Lecture 30: Recursion in MIPS Assembly Language

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand recursion and when it is used.
- Explain the difference between recursion and iteration.
- Understand the concepts of the **base case** and **recursive case**.
- Explain why recursion requires the stack.
- Save and restore `$ra` and registers during recursive calls.
- Write recursive procedures in MIPS.
- Implement recursive factorial and Fibonacci programs.
- Debug recursive programs in the MARS simulator.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how recursion works in MIPS Assembly Language, understand stack frames, implement recursive factorial and Fibonacci functions, and visualize recursive execution using the MARS simulator."**

In the previous lecture, we learned how to write procedures in MIPS, pass parameters using registers, return values, and use the stack to save important information.

Today, we're taking that concept one step further by learning recursion.

Recursion is a technique where a function calls itself to solve a problem by breaking it into smaller versions of the same problem.

Many algorithms in computer science, such as factorial, Fibonacci, tree traversal, and divide-and-conquer algorithms, rely on recursion.

By the end of this lecture, you'll understand how recursive function calls are managed using the stack and how to implement recursive programs in MIPS Assembly Language.

Let's begin.

---

2. What is Recursion?

A **recursive function** is a function that calls itself.

Example in C++:

```
void greet()
{
    greet();
}
```

Question to students:

What happens here?

Answer:

The function keeps calling itself forever, eventually causing a **stack overflow** because there is no stopping condition.

---

3. Base Case and Recursive Case

Every recursive function must have:

1. **Base Case** – stops recursion.
2. **Recursive Case** – calls itself.

Example:

```
int factorial(int n)

{

   if(n == 1)

      return 1;



   return n * factorial(n - 1);

}
```

Explain:

- Base Case → `n == 1`
- Recursive Case → `factorial(n - 1)`

Without the base case, recursion never ends.

---

4. Recursion vs Iteration

Iteration	Recursion

Uses loops	Uses function calls
Usually faster	Slightly slower due to function call overhead
Doesn't always need the stack	Uses the stack for every call
Easier to optimize	Often simpler for recursive problems

---

5. How Recursion Works

Example:

factorial(4)

Execution:

factorial(4)

↓

4 × factorial(3)

↓

4 × (3 × factorial(2))

↓

4 × (3 × (2 × factorial(1)))

↓

1

↓

2

↓

6

↓

24

Emphasize that recursive calls **expand** until the base case is reached, then **return** one by one.

---

6. Why Do We Need the Stack?

Each recursive call has its own:

- Return address
- Parameter value
- Local data

These are stored on the stack.

Visualization:

Top of Stack

factorial(1)

factorial(2)

factorial(3)

factorial(4)

Bottom

When returning:

factorial(1)

↓

factorial(2)

↓

factorial(3)

↓

factorial(4)

This is the **Last In, First Out (LIFO)** behavior of the stack.

---

7. Recursive Factorial Algorithm

Pseudo-code:

Function Factorial(n)

If n == 1

Return 1

Else

Return n × Factorial(n-1)

---

8. MIPS Recursive Factorial

Factorial:

```
addi $sp,$sp,-8
sw   $ra,4($sp)
sw   $a0,0($sp)

li   $t0,1
beq  $a0,$t0,BaseCase

addi $a0,$a0,-1
jal  Factorial

lw   $a0,0($sp)
mul  $v0,$a0,$v0
```

```
    j    Finish

BaseCase:

    li   $v0,1

Finish:

    lw   $ra,4($sp)
    addi $sp,$sp,8

    jr   $ra
```

Walk through this program line by line and explain each stack operation.

---

9. Calling the Recursive Procedure

```
main:

    li $a0,5

    jal Factorial
```

move $t0,$v0

li $v0,10

syscall

Result:

$t0 = 120

---

10. Step-by-Step Stack Trace

For `factorial(3)`:

Call factorial(3)

↓

Push RA

Push 3

↓

Call factorial(2)

↓

Push RA

Push 2

↓

Call factorial(1)

↓

Return 1

↓

Return 2

↓

Return 6

This is one of the most effective diagrams to draw during the lecture.

---

11. Recursive Fibonacci

C++:

```
int fib(int n)

{

    if(n <= 1)

        return n;



    return fib(n-1) + fib(n-2);

}
```

Explain that Fibonacci is recursive but **less efficient** because many values are recalculated.

Visualization:

fib(4)

↓

fib(3) + fib(2)

↓

fib(2)+fib(1)+fib(1)+fib(0)

Use this example to explain repeated computation.

---

12. Recursion Tree

Example:

```
        fib(4)
        /     \
   fib(3)      fib(2)
   /   \      /   \
fib(2) fib(1) fib(1) fib(0)
```

Students quickly see why Fibonacci grows exponentially.

---

13. Visualizing the Stack in MARS

Demonstrate:

1. Set a breakpoint before `jal Factorial`.
2. Click **Step**.
3. Observe:
    - `$sp`
    - `$ra`
    - `$a0`
    - `$v0`
4. Watch `$sp` decrease with each recursive call.
5. Watch `$sp` increase as calls return.

This gives students a clear picture of stack behavior.

---

14. Common Mistakes

Students often:

❌ Forget the base case.

Result:

Infinite recursion

↓

Stack Overflow

❌ Forget to save `$ra`.

Incorrect:

jal Factorial

Correct:

sw $ra,4($sp)

❌ Forget to restore `$ra`.

Correct:

lw $ra,4($sp)

❌ Forget to restore `$sp`.

Always match:

addi $sp,$sp,-8

with:

addi $sp,$sp,8

---

15. Real-Life Analogy

Imagine a stack of books.

You place books one by one:

Book 4

Book 3

Book 2

Book 1

To retrieve them, you remove the top book first:

Book 1

↓

Book 2

↓

Book 3

↓

Book 4

Recursive function calls behave in exactly the same way.

🎤 Lecture Summary

Today, we learned how recursion works in MIPS Assembly Language.

We explored the concepts of base case and recursive case, understood why recursion requires the stack, and implemented recursive procedures such as factorial. We also visualized recursive calls using stack frames and learned the importance of saving and restoring registers during recursive execution.

In the next lecture, we'll begin exploring A**dvanced Stack & Calling Conventions**. We'll learn how to open, read, write, and close files using MIPS syscalls, along with practical examples in the MARS simulator.

Thank you for watching, and I'll see you in the next lecture.
